

TP1

Objectifs :

Il s'agit, pour ce premier TP, de pratiquer les notions vues en classe à propos de la programmation orientée objets.

Pour ce faire, vous devez, pour chacun des programmes incomplets suivants, ajouter ce qui manque pour que le programme compile sans erreurs, s'exécute correctement, puis qu'il affiche ce qu'il est supposé afficher.

Exercice1

Pour le programme suivant :

```
// Ajoutez ce qui manque ici :  
//  
// À partir d'ici, N'AJOUTEZ NI NE MODIFIEZ RIEN  
(c'est interdit) :  
main()  
{  
    cout << "Bonjour" << endl;  
    cout << "Quel est votre nom ?";  
    string nom;  
    cin >> nom;  
    cout << "Bonjour " << nom << endl;  
    return 0;  
}
```

Exercice2

Programmes à compléter

[a.cpp](#)

```
// Ajoutez ce qui manque ici :  
  
// À partir d'ici, N'AJOUTEZ NI NE MODIFIEZ RIEN  
(c'est interdit) :  
  
main()  
  
{
```

```
cout << "Bonjour !" << endl;

cout << "Ce premier programme est très simple
(regardez l'exemple dans l'énoncé)" << endl;

cout << "C'est tout !" << endl;

return 0;

}
```

[b.cpp](#)

```
// Ajoutez ce qui manque ici :

class MaClasse

{

public:

    int x;

    double y;

    MaClasse() {x=0;y=0;}

};

// À partir d'ici, N'AJOUTEZ NI NE MODIFIEZ RIEN
(c'est interdit) :

main()

{

    MaClasse maVariable;

    maVariable.x = 2;

    maVariable.y = 4.5;

    return 0;

}
```

[c.cpp](#)

// Ajoutez ce qui manque ici :

```
class Point
```

```
{
```

```
private:
```

```
    float x;
```

```
    float y;
```

```
public:
```

```
    Point (float monX, float monY)
```

```
    {
```

```
        x=monX;
```

```
        y=monY;
```

```
    }
```

```
    Point () {x=0;y=0;}
```

```
    float getX() {return x;}
```

```
    float getY() {return y;}
```

```
};
```

```
// À partir d'ici, N'AJOUTEZ NI NE MODIFIEZ RIEN  
(c'est interdit) :
```

```
main()
```

```
{  
  
    Point p(2,4);  
  
    Point q(5,6);  
  
    cout << "Coordonnées du premier point : " <<  
p.getX() << "," << p.getY() << endl;  
  
    cout << "Coordonnées du deuxième point : " <<  
q.getX() << "," << q.getY() << endl;  
  
    return 0;  
  
}
```

[d.cpp](#)

```
// Ajoutez ce qui manque ici :  
  
#include<iostream>  
  
class Etudiant  
{  
  
private:  
  
    string Prenom;  
  
    string Nom;  
  
    int Matricule;  
  
public:  
  
    Etudiant() {Prenom=""; Nom=""; Matricule=0;}  
  
    Etudiant(string monPrenom, string monNom, int  
monMatricule)  
  
    {  
  
        Prenom=monPrenom;
```

```
Nom=monNom;

Matricule=monMatricule;

}

void affiche()

{

cout<<"Prenom: "<<Prenom<<"Nom: "<<Nom<<"Matricule:
"<<Matricule<<endl;

}

void setPrenom(string monPrenom)
{Prenom=monPrenom;}

void setMatricule(int monMatricule)
{Matricule=monMatricule;}

};

// À partir d'ici, N'AJOUTEZ NI NE MODIFIEZ RIEN
(c'est interdit) :

main()

{

Etudiant x("Dominic","Genest",97010111);

x.affiche(); // Doit afficher "Prénom : ; Nom : ;
Matricule : "

    x.setPrenom("Thomas");

    x.setMatricule(25010111);

    x.affiche(); // Doit afficher "Prénom : Thomas ;
Nom : Genest ; Matricule : 25010111"

    return 0;

}
```

[e.cpp](#)

```
// Ajoutez ce qui manque ici :

//*****

*****//

// Classe Temps
//

//
//

//*****

*****//

class Temps{

    double m_temps;

public:

    Temps() {m_temps=0;}

    Temps(double tmp)

    {

        m_temps=tmp;

    }

    double getTemps() {return m_temps;}

};

//*****

*****//

// Classe Vitesse
//

//*****

*****//

class Vitesse{
```

```
        double n_vitesse;

public:

    Vitesse() {n_vitesse=0.0; }

    Vitesse(double Vitesse)

    {

        n_vitesse=Vitesse;

    }

    double operator +(double d)

    {

        return (n_vitesse+d);

    }

    double getVitesse() {return n_vitesse;}

};

//*****
//*****//

// Classe Distance
//

//*****
//*****//

class Distance{

private:

    double m_distance;
```

```
public:

    Distance(double Distance)
{m_distance=Distance;}

    double getDistance() {return m_distance;}

};

//*****
//*****//

//    Classe Accélération
//

//*****
//*****//

class Acceleration{

    double m_gravite;

public:

    Acceleration() {m_gravite=0.0;}

    Acceleration(double gravite)

    {

        m_gravite=gravite;

    }

    double getAccel() {return m_gravite;}

    double operator * (Temps t)

    {

        return m_gravite*t.getTemps();

    }

    Distance distanceParcourue(Vitesse V, Temps t)
```

```
{

    double
Distance=m_gravite*t.getTemps()*t.getTemps()/2+V.getV
itesse()*t.getTemps();

    return Distance;

}

};

//*****
//*****//

//    Main
//

//*****
//*****//

// À partir d'ici, N'AJOUTEZ NI NE MODIFIEZ RIEN
(c'est interdit) :

main()

{

    Vitesse vitesseInitiale(10.0); // En mètres par
seconde

    Acceleration gravite(9.81); // mètres par seconde
par seconde (m/s²)

    Temps dureeDeLaChute(25.0); // En secondes

    Vitesse vitesseFinale = vitesseInitiale +
gravite*dureeDeLaChute;

    cout << "Un corps qui chutait initialement à " <<
vitesseInitiale.getVitesse() << " m/s, ";

    cout << "après " << dureeDeLaChute.getTemps() <<
" secondes, ";
```

```
cout << "avec une gravité de " <<
gravite.getAccel() << " m/s/s, ";
```

```
cout << "ira à la vitesse de " <<
vitesseFinale.getVitesse() << " vers le sol (sans
considérer la friction de l'air" << endl;
```

```
cout << "Il aura alors parcouru " <<
gravite.distanceParcourue(vitesseInitiale,dureeDeLaCh
ute).getDistance() << " mètres." << endl;
```

// Remarque : Pour cette dernière ligne, vous avez
probablement deviné que la classe

```
//          "Acceleration" devait comporter une
fonction "distanceParcourue", qui
```

```
//prend en paramètres une vitesse initiale (de type
Vitesse), puis une
```

```
//          durée (de type Temps). Cette
fonction doit retourner un objet d'une
```

```
//          classe qui contient la fonction
"getDistance" (disons la classe Distance).
```

```
//          Cet objet doit contenir la valeur
résultant de la formule suivante :
```

```
//           $d = a \cdot t \cdot t / 2 + v \cdot t$ 
```

```
//          où d est la valeur retournée, a est
l'accélération gravitationnelle,
```

```
//          t est le temps écoulé, et v est la
vitesse initiale.
```

```
//          Cette autre forme de l'équation
donne la même réponse :
```

```
//           $d = t(v + at/2)$ 
```

```
//
```

```
        cout << "Autre calcul : " <<
Acceleration(8.4).distanceParcourue(Vitesse(-
5.0),Temps(20.0)).getDistance() << endl;

        return 0;

}
```